

变物性与收缩边界下圆柱药材的热湿迁移建模

摘要

针对圆柱药材烘干中的时变边界、状态相关物性和几何收缩，建立径向热湿有限体积模型。环境与半径数据用 PCHIP 重构，内部采用细网格与全隐式 Picard 迭代；对含水率相关扩散率使用 Kirchhoff 积分型界面通量，并将计算解独立采样到题目要求的时空位置。

问题一：使用常热物性和 $D(C)$ ，计算 1800 s 内的逐秒温湿场。末时中心、表面温度为 33.5770、36.7865 °C，含水率为 2.5500、1.5103 kg/kg，显示短时失水主要发生在表层。

问题二：引入题给 $\rho(C), c_p(C), k(C), D(T, C)$ ，通过状态相关系数形成耦合。3 h 时中心、表面温度为 49.8493、49.9666 °C，含水率为 1.7662、1.0081 kg/kg；热场已接近均匀，水分梯度仍明显。

问题三：以全截面最大含水率首次不超过 0.15 kg/kg 为判据，线性定位相邻时间层的数值事件，得到 **57.1776 h**。该结果以 4 h 后环境保持末值为条件，逐分钟输出与事件估计分别处理。

问题四：在均匀径向材料收缩假设下，以 $\xi = r/R(t)$ 固定材料坐标。材料速度与网格速度相同，相对对流项相消，保留 R^{-2} 扩散尺度与 R^{-1} 表面尺度。采用附录 4 物性得到 **50.8275 h**，达标半径 1.2000 cm；同物性固定半径对照为 129.1011 h，条件性缩短 60.63%。

空间加密、分阶段时间步检验、干基质量平衡及长期二维对照分别用于量化数值与降维误差。模型明确区分经验热物性密度与干物质质量权重；蒸发潜热、界面吸湿平衡与局部变形未由题给数据闭合，作为适用范围限制。

关键字： 药材烘干 非线性扩散 有限体积法 材料收缩 首达时间

一、问题重述

1.1 问题背景

药材烘干的目标并非简单地使表面水分降低，而是在不过度加热和不过度干燥表层的前提下，使药材内部所有位置均达到规定含水标准。烘房温度升高后，热量由外向内传递，表层水分同时在内部浓度梯度和表面对流传质的共同作用下向外迁移。由于传热和传质的特征时间不同，药材内部会长期存在“表面已干、中心仍湿”的空间不均匀现象。因此，只记录环境状态或表面状态不能直接判断整根药材是否完成烘干，必须建立能够描述内部时空分布的非稳态模型。

题目研究对象为长度 25 cm、初始半径 2 cm 的圆柱形药材，其初始温度为 28 °C，初始水分浓度为 2.55 kg/kg。附件一给出前 4 h 烘房温度和水分浓度的离散测量值，附件二给出 72 h 内药材半径的变化记录。四个问题依次加入状态相关物性、全截面达标约束和几何收缩效应，实质上构成由固定区域线性问题向移动区域非线性问题逐层推进的建模任务。

1.2 问题提出

根据所给几何、初始条件、环境序列和物性关系，需要获得药材温度、水分浓度随时间及径向位置的分布，并确定内部最难干燥位置首次满足品质阈值的时刻。为使四问结果具有统一口径，还需要明确环境数据的插值方式、表面传递边界、空间离散方法、终止事件以及收缩后“固定物理位置”的有效范围。

问题一：在半径和热物性保持不变的基准情形下，处理时变环境与药材表面的热、质交换，计算预热阶段 0–1800 s 内的径向温度和水分浓度，并形成每 1 s、每 0.1 cm 的完整结果。该问用于建立后续各问共享的控制方程、边界条件和数值离散框架。

问题二：在问题一基础上，将密度、比热容、导热系数及水分扩散系数改写为温度和水分浓度的函数。模型由常系数扩散方程转为状态耦合的非线性方程，要求求得 0–3 h 的温湿场，并分析物性变化如何影响中心与表面的响应差异。

问题三：沿用问题二的变物性模型，将计算时域延长至药材内部所有位置的水分浓度均不超过 0.15 kg/kg。该问的关键不是某个离散输出时刻是否达标，而是确定全空间最大水分浓度首次跨越阈值的连续时间，并给出每 60 s 的完整结果与每 6 h 的典型剖面。

问题四：进一步考虑药材半径随时间收缩，并使用附录 4 给出的另一组状态相关物性。计算域随时间变化，固定半径采样点还可能在后期落到药材外部。因此，需要建立

材料随动的移动边界模型，重新确定达标时长，规范动态表面及固定物理位置的输出，并以同物性固定半径计算评估收缩的影响。

二、问题分析

四问共享圆柱径向传热、传质方程与表面 Robin 边界，但依次加入变物性、全域阈值及几何收缩。问题一的热物性为常数，水分扩散系数仍随含水率变化；问题二进一步通过物性构成温湿双向状态耦合；问题三采用最大含水率的首达事件；问题四在材料随半径同比例收缩的假设下转为固定材料坐标。

问题一、二要求逐秒输出，问题三、四要求逐分钟输出。这些是提交文件的采样要求，不是内部计算网格必须采用的间隔。由于低含水率表层的扩散率变化剧烈，必须使用更细的内部网格，并将解采样至题目指定位置。空间精度、时间精度、阈值插值和文件采样应分别检验。

问题四与问题三采用不同物性，二者时间差不能直接解释为收缩贡献。因此设置采用附录 4 物性、相同环境而保持初始半径的对照模型。另用长期二维轴对称模型检验端面对全域首达时间的影响。



图 1 数据、模型、收敛验证与结果输出之间的关系

三、模型假设

1. 药材为各向同性连续介质，初始温度、干基含水率均匀；环境关于轴线对称。主模型描述中截面的径向分布，轴向端面影响另用长期二维计算检验。
2. 内部热量按 Fourier 定律传递，水分按题给有效扩散系数迁移。忽略辐射、托盘接触传热、宏观渗流及各向异性。
3. 四问沿用附录 2 的 $h_T = 25 \text{ W}/(\text{m}^2\text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。题给环境含湿量作为 Robin 边界的有效驱动值；这是一种题目数据层面的闭合，不声称空气与药材的 kg/kg 含湿量具有相同物理分母。实物标定需要吸湿等温线或相应界面关系。
4. 题给 $\rho(C)c_p(C)$ 作为经验体积储热系数使用，热湿耦合通过这些物性及 $D(T, C)$ 表达。未显式加入蒸发冷却；缺少界面相变通量、吸湿关系及相关热力学闭合时，不从现有附件唯一反演该项。
5. 附件 1 覆盖 $0 \sim 14400 \text{ s}$ ，其后环境保持末值 50.165°C 、 0.04986 kg/kg 。长期预测以这一运行情景为条件，另检验温度、含湿量、传质系数、扩散倍率与半径的不确定性。
6. 问题四长度固定，材料速度为 $v_r = (\dot{R}/R)r$ ，各材料点同比例径向收缩。干物质无损失，初始干物质密度均匀，故其当前体积密度按 $\rho_d(t) = \rho_{d0}(R_0/R)^2$ 变化。 ρ_d 是质量平衡的基准，不与经验热物性中的 $\rho(C)$ 混用。

上述模型没有同时强制 $\rho(C) = \rho_d(t)(1 + C)$ 。给定半径、经验密度和均匀变形未必满足该恒等式；本文把题给密度关系解释为热方程的经验闭合，并明确限制其物理适用范围。若要求严格的混合物密度与能量守恒，应进一步建立局部孔隙率、变形和相变模型。

四、符号说明

表 1 主要符号及单位

符号	含义	单位
r, z	径向、轴向坐标; $z = 0$ 为中截面	m
$R(t), L$	当前半径、长度	m
T, T_e	药材、环境温度	°C
C	药材干基含水率	kg 水/kg 干物质
C_e	题给环境有效含湿驱动值	kg/kg
ρ, c_p, k	经验密度、比热容、导热系数	kg/m ³ , J/(kg K), W/(m K)
ρ_d	单位当前体积的干物质质量	kg 干物质/m ³
D	有效水分扩散系数	m ² /s
h_T, h_m	表面换热、有效传质系数	W/(m ² K), m/s
ξ	材料坐标 $r/R(t)$	1
v_r, w_r	材料、网格径向速度	m/s
V_i, A_f	单位长度控制体体积、界面面积	m ² , m
t_d	最大含水率达到阈值的首达估计	s

扩散系数中的绝对温度为 $T_K = T + 273.15$; 温差的摄氏度与开尔文数值相同。本文规定时间首达误差与场量误差分别报告, 不以提交表的四位小数格式代替精度说明。

五、模型准备与统一数值方法

5.1 输入数据与边界重构

圆柱初始半径 $R_0 = 0.02$ m、长度 $L = 0.25$ m, 初始状态为

$$T(r, 0) = 28 \text{ } ^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg.} \quad (1)$$

附件 1 包含 241 个环境时刻, 间隔 60 s; 附件 2 包含 145 个半径时刻, 间隔 1800 s。采用 PCHIP 保形插值 [3], 环境在 4 h 后保持末值, 半径仅在附件覆盖的 72 h 内使用。环境测量波动不强行单调化; 半径按非增序列重构, 调整数记录在数据审计文件中。

5.2 固定域控制方程与边界

固定半径下采用

$$\rho(C)c_p(C)\frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rk(C)\frac{\partial T}{\partial r}\right), \quad (2)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rD(T, C)\frac{\partial C}{\partial r}\right). \quad (3)$$

该有效扩散模型以均匀干物质参考权重定义含水率储存；题给经验密度只进入热方程，所涉及的近似已在模型假设中说明。轴线为零通量，侧表面为

$$\partial_r T(0, t) = \partial_r C(0, t) = 0, \quad (4)$$

$$-k\partial_r T(R, t) = h_T(T_s - T_e), \quad (5)$$

$$-D\partial_r C(R, t) = h_m(C_s - C_e). \quad (6)$$

以向外通量为正，环境更热时热流指向内部，药材更湿时水分流向外部。

5.3 题给物性与状态耦合

附录 2 给出第一问的常热物性及非线性扩散系数：

$$\rho = 820, \quad c_p = 2600, \quad k = 0.36, \quad D = 7 \times 10^{-9} e^{-0.89/C}. \quad (7)$$

第二、三问使用附录 3：

$$\begin{aligned} \rho &= 650 + 128C, & c_p &= 1450 + 2736 \frac{C}{C+1}, \\ k &= 0.21 + 0.38 \frac{C}{C+1}, & D &= 2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/(T+273.15)}. \end{aligned} \quad (8)$$

第四问使用附录 4：

$$\begin{aligned} \rho &= 760 + 90C, & c_p &= 1850 + 2150 \frac{C}{C+1}, \\ k &= 0.12 + 0.20 \frac{C}{C+1}, & D &= 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/(T+273.15)}. \end{aligned} \quad (9)$$

含水率决定储热与导热物性，温度和含水率共同决定有效扩散率，构成状态耦合。

5.4 守恒控制体与非线性界面通量

内部求解使用 321 个径向节点，取表面加密映射 $\xi_i = 1 - [1 - i/(N-1)]^2$, $i = 0, \dots, N-1$ ，并令 $r_i = R(t)\xi_i$ ；固定域取 $R(t) = R_0$ 。题目要求的 21 个物理位置是

输出网格，不限制内部离散。各界面采用实际相邻节点距离 $r_{i+1} - r_i$ 计算传导系数 $G_f = A_f k_f / (r_{i+1} - r_i)$ 或 $A_f D_f / (r_{i+1} - r_i)$ 。界面在相邻节点中点，中心及表面控制体边界分别截在 0 和当前半径：

$$V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2), \quad A_f = 2\pi r_f. \quad (10)$$

以 ϕ 表示 T 或 C ，全隐式控制体方程为

$$\frac{s_i V_i}{\Delta t} (\phi_i^{n+1} - \phi_i^n) = G_E (\phi_{i+1}^{n+1} - \phi_i^{n+1}) - G_W (\phi_i^{n+1} - \phi_{i-1}^{n+1}), \quad (11)$$

热方程取 $s = \rho c_p$ ，含水率方程取 $s = 1$ 。表面直接加入 $A_R h(\phi_e - \phi_s)$ 。内部界面通量在相邻控制体中符号相反，因此离散求和能消去内部交换 [2]。

对 k 用相邻节点平均；对变化剧烈的 $D(T, C) = b(T)e^{-a/C}$ ，使用 Kirchhoff 原函数

$$F_a(C) = Ce^{-a/C} + a \text{Ei}(-a/C), \quad F'_a(C) = e^{-a/C}, \quad (12)$$

构造界面扩散率

$$D_f = b(T_f) \frac{F_a(C_{i+1}) - F_a(C_i)}{C_{i+1} - C_i}, \quad T_f = \frac{T_i + T_{i+1}}{2}. \quad (13)$$

当两浓度相近时使用中点极限。该格式对浓度方向积分，但仍近似界面温度和空间几何，不能称为原方程的精确离散。其作用是避免低含水率薄层中简单算术平均过度放大界面扩散率，效果由后文加密试验验证。

5.5 非线性迭代、时间推进与事件

每一步由旧解开始 Picard 迭代：由当前含水率求热物性及新温度，再更新界面扩散率并求含水率。当温度迭代差小于 10^{-8}°C 、含水率迭代差小于 10^{-10} kg/kg 时结束，最多 30 次，失败则报错。第一、二问时间步为 1 s；第三、四问前 3 h 也为 1 s，此后为 10 s。步长在边界延拓切换、输出时刻处截断，避免跨越数据处理节点。第二、三问同网格、同物性、同前段步长，因此其共同时间点可直接核对。

定义 $M(t) = \max_i C_i(t)$ ，若 $M^n > 0.15 \geq M^{n+1}$ ，令

$$\lambda = \frac{M^n - 0.15}{M^n - M^{n+1}}, \quad \hat{t}_d = t_n + \lambda \Delta t. \quad (14)$$

同时插值达标剖面。该量是离散数值解的事件估计，不能保证真实连续解的阈值恰落在这两个时间层之间；总误差还包括空间、时间和模型误差。提交文件另按题意逐秒或逐分钟采样，最终完整分钟覆盖估计事件。

六、问题一：常热物性预热模型与结果

6.1 特征尺度与求解设置

由题给物性，热扩散率 $\alpha = k/(\rho c_p) = 1.689 \times 10^{-7} \text{ m}^2/\text{s}$ ；初始水分扩散率 $D_0 = 4.938 \times 10^{-9} \text{ m}^2/\text{s}$ 。以半径为尺度， $R_0^2/\alpha \approx 2369 \text{ s}$ ， $R_0^2/D_0 \approx 8.10 \times 10^4 \text{ s}$ ，说明短时热扰动比水分扰动传播得快。对应 $h_T R_0/k = 1.389$ 、 $h_m R_0/D_0 = 3.240$ ，内部梯度不可忽略。第一问保持密度、比热容与导热系数为常数，水分方程仍通过 $D(C)$ 非线性求解。每 1 s 积分，采样到 0, 0.1, ..., 2.0 cm。中心控制体为完整小圆盘，并非半圆盘。

6.2 温度与含水率结果

表 2 问题 1 温度分布 (°C)

t/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0004	28.0041	28.0324	28.1802
300	28.0414	28.0642	28.1522	28.3683	28.8495
600	28.4549	28.5376	28.8053	29.3168	30.1650
900	29.3262	29.4602	29.8774	30.6177	31.7305
1200	30.5447	30.7117	31.2241	32.1143	33.4304
1500	31.9976	32.1887	32.7681	33.7486	35.1218
1800	33.5770	33.7736	34.3654	35.3631	36.7865

表 3 问题 1 含水率分布 (kg/kg)

t/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2475
300	2.5500	2.5500	2.5500	2.5492	2.0511
600	2.5500	2.5500	2.5500	2.5352	1.8771
900	2.5500	2.5500	2.5497	2.5045	1.7549
1200	2.5500	2.5500	2.5482	2.4645	1.6587
1500	2.5500	2.5499	2.5445	2.4206	1.5788
1800	2.5500	2.5497	2.5383	2.3755	1.5103

1800 s 时中心与表面温度分别为 33.5770、36.7865 °C，含水率分别为 2.5500、1.5103 kg/kg。温度由外向内提高，水分主要从外层逸出。表层变干后 $e^{-0.89/C}$ 下降，形成低扩散率区域，解释了中心含水率在预热阶段仍接近初始值。题目指定七个时刻的表

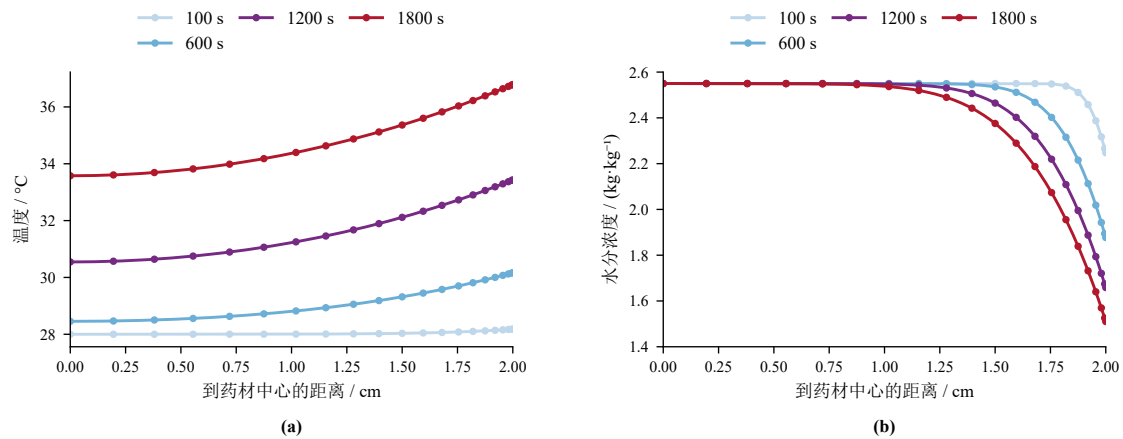


图2 问题一的径向温度与含水率分布

格由终算数据生成，导出程序使用同一数据生成 result1.xlsx；其格式含温度、水分浓度两表，时间为 1–1800 s。四位小数遵从题目格式，数值误差另列于模型检验。

七、问题二：变物性热湿耦合模型与结果

7.1 耦合路径

将附录 3 关系代入统一框架，在每一时间层更新热物性和有效扩散率。含水率降低使储热能力变化，温度升高提高 Arrhenius 项；与此同时，低含水率会使 $e^{-0.45/C}$ 快速下降。因此温度趋于均匀不代表内部含水率已经达标。

表4 问题2 温度分布 (°C)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5	32.1911	32.3839	32.9674	33.9620	35.4141
1.0	40.3822	40.5542	41.0609	41.8784	43.0001
1.5	45.8464	45.9345	46.1927	46.6049	47.1393
2.0	48.4498	48.4876	48.5980	48.7732	49.0030
2.5	49.4667	49.4789	49.5133	49.5645	49.6617
3.0	49.8493	49.8552	49.8746	49.9100	49.9666

7.2 结果解释与输出

3 h 时中心与表面温度分别为 49.8493、49.9666 °C，含水率分别为 1.7662、1.0081 kg/kg。此时温差很小，中心与表面的含水率差仍明显。长期阶段主要需要等待内部水分穿过已较干的外层。result2.xlsx 包含 1–10800 s、21 个题目指定物理位置的温

表 5 问题 2 含水率分布 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5	2.5499	2.5489	2.5255	2.3257	1.6487
1.0	2.5257	2.4947	2.3578	2.0230	1.4712
1.5	2.3860	2.3256	2.1344	1.8020	1.3476
2.0	2.1708	2.1084	1.9236	1.6260	1.2311
2.5	1.9567	1.9006	1.7360	1.4720	1.1167
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

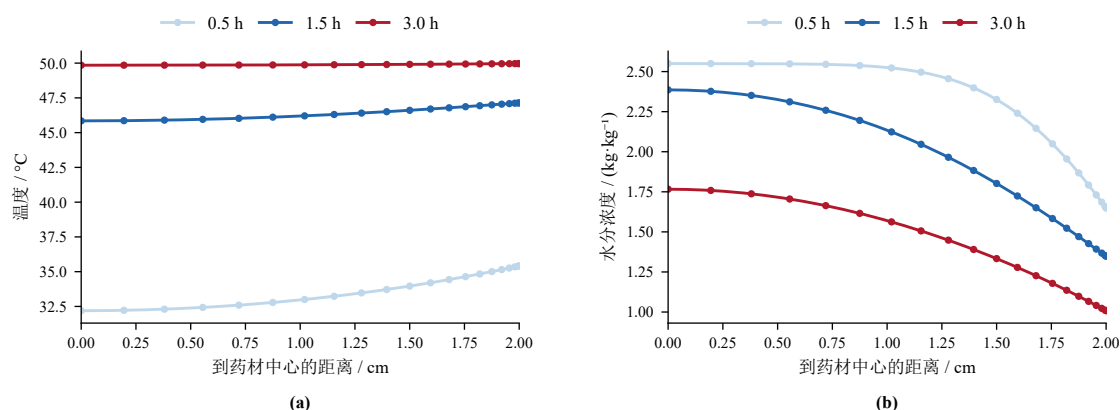


图 3 问题二状态相关物性下的温度与含水率分布

度和水分浓度。内部使用细网格计算，再采样到输出位置。第三问前 3 h 使用完全相同的时间推进和物性；共同输出时刻的最大差在核验文件中直接报告。

八、问题三：全截面达标时间

8.1 全域判据与控制位置

题目要求药材各处含水率低于 0.15 kg/kg。连续下降时用等于阈值的交叉时刻作为边界时间，定义

$$t_d = \inf\{t \geq 0 : \max_{0 \leq r \leq R_0} C(r, t) \leq 0.15\}. \quad (15)$$

程序每个内部时间层保留全网格最大值判据，并检查相邻含水率是否由内向外非增；中心控制是计算核验结果，不是仅由矩阵性质预先推出。径向离散仍不能自动保证所有连续空间点的精度，故另进行空间加密。

8.2 达标时间与典型剖面

在恒定末端环境延拓下，计算得到

$$\hat{t}_d = 205839.3726 \text{ s} = 57.1776 \text{ h.} \tag{16}$$

中心在达标剖面中为 0.1500 kg/kg，表面为 0.0525 kg/kg。图中表面曲线较早接近低含

表 6 问题 3 每 6 h 及达标时含水率 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6	1.0157	0.9870	0.9007	0.7548	0.5336
12	0.4549	0.4420	0.4021	0.3290	0.1640
18	0.2980	0.2907	0.2680	0.2247	0.0842
24	0.2371	0.2321	0.2162	0.1851	0.0663
30	0.2053	0.2013	0.1887	0.1637	0.0597
36	0.1854	0.1821	0.1714	0.1501	0.0565
42	0.1717	0.1687	0.1594	0.1404	0.0547
48	0.1615	0.1588	0.1504	0.1332	0.0536
54	0.1535	0.1511	0.1434	0.1275	0.0528
57.1776	0.1500	0.1477	0.1402	0.1249	0.0525

湿环境，说明仅根据表面判断会提前停机。随着含水率下降，有效扩散率中的 $e^{-0.45/C}$ 项减小，后期下降速度放缓。

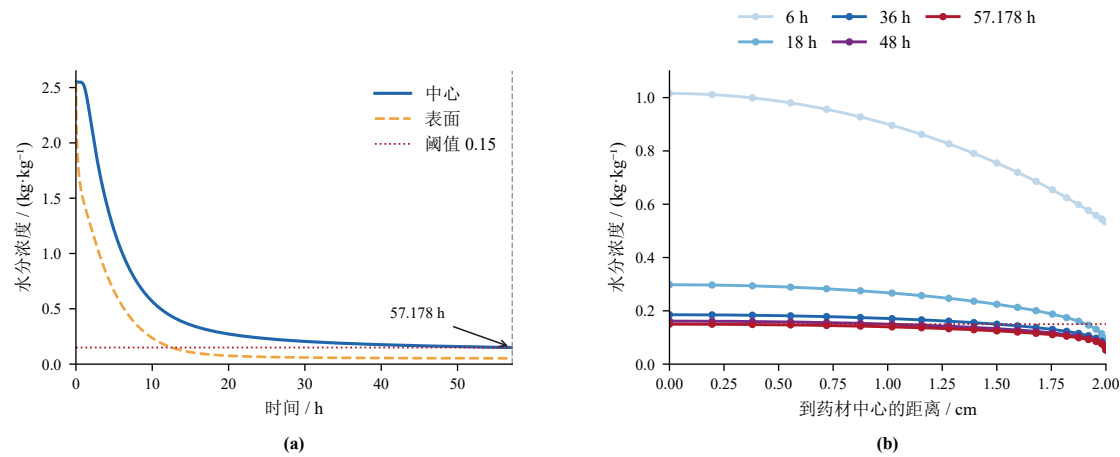


图 4 问题三的最大含水率首达过程及典型剖面

8.3 结果文件与适用范围

result3.xlsx 从 60 s 起逐分钟记录至 205860 s，末行覆盖数值首达估计，论文末行则为事件插值剖面。首达估计与输出时刻不同，不能把末行时间直接当成更精确的达标时间。主答案使用一维中截面模型，长期二维全体积结果及其偏差另在检验节给出。

九、问题四：材料收缩模型与达标时间

9.1 质量基准和材料坐标

使用附件 2 重构 $R(t)$ ，令 $\xi = r/R(t)$ 。均匀径向收缩假设给出材料速度 $v_r = \dot{R}\xi$ ，网格速度同为 $w_r = \dot{R}\xi$ 。干物质与水分守恒写为

$$\partial_t \rho_d + \frac{1}{r} \partial_r (r \rho_d v_r) = 0, \quad (17)$$

$$\rho_d (\partial_t C + v_r \partial_r C) = \frac{1}{r} \partial_r (r \rho_d D \partial_r C). \quad (18)$$

初始 ρ_d 均匀时， $\rho_d(t)R^2(t) = \rho_{d0}R_0^2$ ，故 ρ_d 在每个时刻对空间均匀，可从 (18) 中约去。这样固定的材料控制体具有固定干物质质量， C 仍是水质量与干物质质量之比。

设 $\psi(\xi, t) = C(R(t)\xi, t)$ ，则

$$\partial_t C|_r = \partial_t \psi|_\xi - \frac{\dot{R}}{R} \xi \partial_\xi \psi, \quad v_r \partial_r C = \frac{\dot{R}}{R} \xi \partial_\xi \psi. \quad (19)$$

两项相消，得到

$$\partial_t C|_\xi = \frac{1}{R^2 \xi} \partial_\xi (\xi D \partial_\xi C), \quad (20)$$

$$\rho(C) c_p(C) \partial_t T|_\xi = \frac{1}{R^2 \xi} \partial_\xi (\xi k \partial_\xi T). \quad (21)$$

这里不能再单独加入 $-(\dot{R}/R)\xi \partial_\xi C$ ，否则相当于把网格运动当作材料与网格之间的相对运动。温度方程仍采用题给经验储热系数，并未由 (17) 重新解释为严格混合物能量守恒。

9.2 边界、离散与质量核验

表面条件变为

$$-\frac{k}{R} \partial_\xi T = h_T (T_s - T_e), \quad -\frac{D}{R} \partial_\xi C = h_m (C_s - C_e). \quad (22)$$

内部扩散尺度为 R^{-2} ，表面项尺度为 R^{-1} 。采用固定材料节点、Kirchhoff 界面通量及全隐式迭代，不需迎风项，也不使用 Courant 数作为精度证据。记单位总干质量归一化的控制体权重为 $\omega_i = V_i / \sum_j V_j$ ，则每步检查

$$\varepsilon_n = \sum_i \omega_i (C_i^{n+1} - C_i^n) + \Delta t \frac{2h_m}{R^{n+1}} (C_s^{n+1} - C_e^{n+1}). \quad (23)$$

累计残差除以初始平均含水率，形成可比较的守恒指标。该检查与材料坐标的干物质质量权重一致。



图 5 材料坐标收缩求解与验证流程

9.3 结果与同物性固定半径对照

计算得到

$$\hat{t}_d = 182979.0811 \text{ s} = 50.8275 \text{ h}, \quad R(\hat{t}_d) = 1.2000 \text{ cm}. \quad (24)$$

表中固定物理位置通过 $\xi = r_j / R(t)$ 插值；当 $r_j > R(t)$ 时该位置位于材料外，留空；末列始终为 $\xi = 1$ 的实时表面。result4.xlsx 保留 0, 0.1, ..., 1.9 cm 及动态表面，逐分钟输出至 183000 s。

采用相同附录 4 物性、相同环境及相同数值设置，仅保持 $R = 2 \text{ cm}$ 的对照时间为 129.1011 h。收缩情景相对于该对照缩短 60.63%。这一比例是本模型及给定收缩曲线下的条件性比较，不代表对其他材料或收缩机理的普遍规律。扩散距离缩短解释了主要加

表 7 问题 4 每 6 h 及达标时含水率 (kg/kg)

t/h	0 cm	0.5 cm	1.0 cm	动态表面
6	1.7165	1.5350	1.0213	0.4216
12	0.7342	0.6517	0.4069	0.1666
18	0.4066	0.3668	0.2388	0.0889
24	0.2838	0.2599	0.1783	0.0671
30	0.2253	0.2085	0.1488	0.0593
36	0.1921	0.1790	0.1313	0.0558
42	0.1706	0.1598	0.1196	0.0539
48	0.1556	0.1463	0.1113	0.0529
50.8275	0.1500	0.1413	0.1081	0.0525

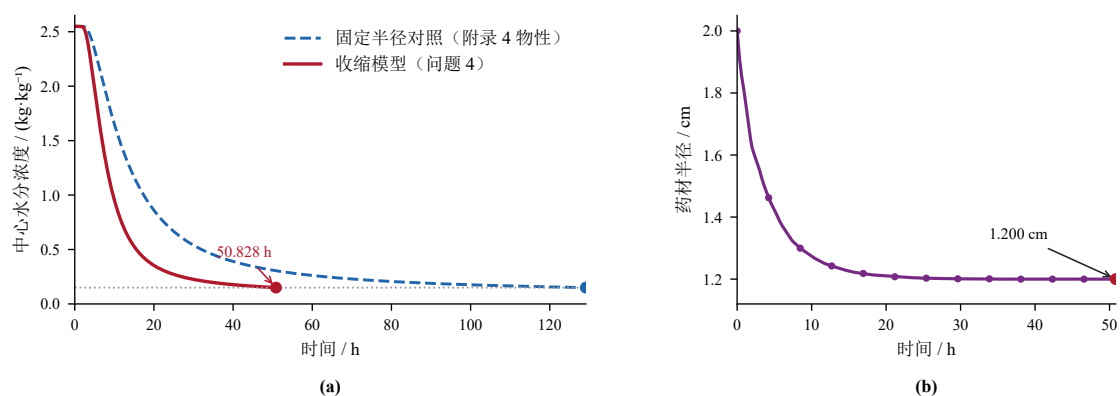


图 6 同物性固定半径与材料收缩情景的比较

速方向。由于第三问采用另一组物性，不能将其与第四问的时间差全归因于收缩。本文也不再把纯坐标项的增删解释为独立的物理收缩机制。

十、模型检验、评价与推广

10.1 空间与时间收敛

表 8 表面加密网格下的首达时间 (h)

节点数	问题三	问题四
81	57.178394	50.828346
161	57.177762	50.827687
321	57.177603	50.827523
641	57.177564	50.827481

该试验保持前 3 h 的 1 s 步长和后续 10 s 步长不变，只改变内部节点数。本文选择 321 节点，以更细网格作对照；网格差异不是相对于真解的严格误差界。时间检验保持

表 9 分阶段时间步变化的首达检验

情形	基准/h	后段 5 s/h	后段差/s	前段减半差/s
问题 3	57.177603	57.173549	14.597	0.108
问题 4	50.827523	50.824138	12.183	0.034

空间网格不变，将 3 h 后的 10 s 减为 5 s；另将前 3 h 的 1 s 减为 0.5 s 检查早期时间误差。不能将后段步长检验误称为全程减半。首达估计同时受空间、时间及模型误差影响，输出四位小数仅为题目格式。第一、二问还比较 321 与 641 节点在共同物理位置的场量

表 10 场量网格差与前段一致性

比较量	温度最大差/ $^{\circ}\text{C}$	含水率最大差/kg/kg
问题 1: 321 与 641 节点	4.31×10^{-6}	1.47×10^{-5}
问题 2: 321 与 641 节点	4.41×10^{-6}	1.29×10^{-5}
问题二、三共同时间点	0	0

差异。第二、三问前段状态在共同输出时刻作直接核对，防止把物理模型一致误写为数值序列一致。

10.2 守恒与独立基准

质量残差按 (23) 使用同一时间层边界通量计算；浮点量级残差说明离散总量平衡良好，但不是实际物理模型正确的充分条件。另以常扩散率圆柱 Robin 边界的解析模态

$$u(r, t) = J_0(\beta r/R) e^{-D\beta^2 t/R^2}, \quad h = \frac{D\beta J_1(\beta)}{R J_0(\beta)} \quad (25)$$

表 11 归一化干基质量平衡残差

情形	最大单步残差/kg/kg	相对累计残差
问题 1	1.78×10^{-15}	9.46×10^{-14}
问题 2	2.74×10^{-15}	1.39×10^{-12}
问题 3	1.86×10^{-14}	3.02×10^{-13}
问题 4	3.70×10^{-15}	6.19×10^{-13}

核验圆柱几何和边界处理, 取 $\beta = 1, D = 10^{-7} \text{ m}^2/\text{s}$ 、 $R = 0.02 \text{ m}$ 、 600 s 终止时间。161 节点、 0.5 s 步长的最大绝对差为 8.34×10^{-6} 。Kirchhoff 界面系数另与直接数值积分比较, 相对差为 1.48×10^{-12} ; 封闭均匀材料在给定收缩下含水率保持不变, 误差为 3.55×10^{-15} 。

10.3 长期二维轴对称验证

在 $0 \leq \xi \leq 1, 0 \leq z \leq L/2$ 的半圆柱上计算, $z = 0$ 为中截面对称面, $z = L/2$ 为暴露端面。径向外表面和暴露端面施加 Robin 条件; 第四问只在径向随材料收缩。二维水分通量使用与一维相同的 Kirchhoff 关系, 采用独立稀疏矩阵组装。比较使用相同径向

表 12 二维全域与同设置一维首达比较 (前段 10 s)

问题	径向 $\times$ 半轴向	后段/s	一维/h	二维/h	差/s
3	41 $\times$ 31	60	57.22198	57.21976	-7.995
4	41 $\times$ 31	60	50.86462	50.86461	-0.029
3	41 $\times$ 61	60	57.22198	57.21987	-7.600
4	41 $\times$ 61	60	50.86462	50.86461	-0.028

网格、时间设置的一维和二维首达事件, 半轴向节点从 31 加密至 61。原先仅到 1800 s 的短期结果不能替代这项长期检查。二维径向网格为 41 节点、前段 10 s 和后段 60 s , 此处估计的是这些数值设置下的端面影响; 若需对数秒量级差异作精确判断, 还需进一步细化二维径向网格与时间步。本文的一维空间、时间检验已单独报告, 不能代替二维模型的全面收敛分析, 更不作为共享物性假设的实验验证。

10.4 参数与环境敏感性

温度扰动直接写为 $\pm 2^\circ\text{C}$, 不对摄氏读数计算百分比。采用

$$S_T = \frac{t_d(T+2) - t_d(T-2)}{4t_d(T)} \quad (\text{K}^{-1}) \quad (26)$$

表 13 敏感性试验 (81 节点; 前段 2 s、后段 20 s)

情景	问题三/h	问题四/h
同设置基准	57.1866	50.8351
末端温度 $-2\text{ }^{\circ}\text{C}$	60.9788	54.1595
末端温度 $+2\text{ }^{\circ}\text{C}$	53.7184	47.7913
末端含湿量 -5%	57.1392	50.7833
末端含湿量 $+5\%$	57.2385	50.8939
扩散倍率 -5%	59.8468	53.1872
扩散倍率 $+5\%$	54.7854	48.7099
传质系数 -5%	57.5314	50.9703
传质系数 $+5\%$	56.8810	50.7173
收缩幅度 -5%	57.1866	53.7559
收缩幅度 $+5\%$	57.1866	48.0205

衡量单位温升的相对时间变化; 对含湿量、扩散倍率和传质系数可计算相对参数弹性。半径扰动通过缩放收缩幅度保留初始 2 cm 半径, 避免改变题目初始几何。所有情景保持同一网格与步长, 分别对照同设置的基准。灵敏度只说明所测试范围内的影响, 不据少数参数宣布全局最重要控制量。4 h 以后恒定延拓仍是情景假设, 不能由前 4 h 数据唯一确定。

10.5 优点、局限与应用

本文统一四问的输入、物性、边界和输出, 利用材料质量权重处理收缩, 并分别检查空间、时间、守恒与二维降维误差。全部数值表格由结果文件生成, 避免正文与附件手工更新不同步。主要局限有三: 其一, 题给空气含湿量直接作为有效 Robin 驱动, 缺少实测吸湿等温线; 其二, 未显式包含潜热、内部蒸发与严格混合物能量闭合; 其三, 同比分布的收缩及各向同性扩散不描述局部孔隙变化、开裂或轴向缩短。因而结果应视为题给经验关系下的预测, 不能声称已通过实际药材实验标定。工程上可结合内部含水率测量校正有效扩散率与表面传质关系, 再用于停机判断。能耗、品质和温度上限尚未进入优化目标, 本文不直接给出未经验证的最优烘干制度。

10.6 与三篇参考文献的对照及后续改进

da Silva 等 [6] 采用圆柱有限体积模型, 将水分损失与表面汽化潜热耦合, 并比较不同热方程简化。在其 $58.6\text{ }^{\circ}\text{C}$ 香蕉试验中, 忽略潜热使温度拟合的 R^2 从 0.98843 降至 0.75502。这是特定材料的实验结果, 不能直接换算成本题误差, 却表明不应仅以题面未

给潜热为由认定蒸发冷却可以忽略。

后续可在完成界面含湿关系与干物质质量标定后，比较无潜热和有潜热两个模型。以向外水质量通量 $j_w > 0$ 表示蒸发，则表面能量平衡为

$$-k\partial_n T = h_T(T_s - T_e) + L_v(T_s)j_w, \quad j_w = \rho_d h_m(C_s - C_{eq}). \quad (27)$$

式中 L_v 可先使用经温标核验的自由水近似，但 C_{eq} 应来自材料平衡吸湿关系。还需连同水分携带的焓和经验储热关系一起检查，不能仅将 $\rho c_p \partial_t T$ 机械替换为 $\partial_t(\rho c_p T)$ 。本次主结果未引入该扩展，避免把尚未标定的模型当作已验证答案。

张继凯等 [7] 对 6 mm 山药片的红外联合热风干燥进行了收缩与非收缩对照，并以实测温度、含水率和品质指标验证。这支持保留本题的同物性几何对照；但本题没有红外热源、局部面积修正系数和品质测量，因此不直接加入其辐射项、不重复叠加其经验面积修正，也不采用其 60 °C 品质结论作为本题最佳温度。

Tzempelikos 等的会议稿 [8] 使用厚度方向一维模型和外部流场计算得到的迎风、背风传递系数，含水率方程显式区分干固体密度与体积密度。该文提示应辨明几何主导方向、质量基准和外部边界；其 Halsey 吸湿关系及 quince 物性不移植到本题药材。若补充环境相对湿度、风速及内部实验数据，可再标定界面关系并检验恒定传递系数假设。

三篇文献提供了扩展模型和实物验证的依据；本文空间加密、守恒残差和二维对照属于数值验证，不能等同于这些文献的实验拟合证据。

AI 工具使用声明

本稿使用 AI 工具辅助题意整理、建模推导、程序修改、数值验证、图表生成、参考文献阅读和文字修改。题目附件由程序读取，本文数值结果由本地求解器计算，并通过空间与时间加密、质量平衡、解析模态及二维轴对称对照检查。上述检查属于程序执行的数值核验，不等同于实测验证或参赛队已经完成人工复核。正式提交前，参赛队应核对模型假设、程序与附件，确认工具使用记录，并对最终内容承担责任。

参考文献

- [1] Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.
- [2] Patankar S V. Numerical Heat Transfer and Fluid Flow[M]. Washington, DC: Hemisphere, 1980.
- [3] Fritsch F N, Carlson R E. Monotone piecewise cubic interpolation[J]. SIAM Journal on Numerical Analysis, 1980, 17(2): 238–246. DOI: 10.1137/0717021.
- [4] Luikov A V. Systems of differential equations of heat and mass transfer in capillary-porous bodies[J]. International Journal of Heat and Mass Transfer, 1975, 18(1): 1–14.
- [5] Adrover A, Brasiello A. A moving boundary model for food isothermal drying and shrinkage: One-dimensional versus two-dimensional approaches[J]. Journal of Food Process Engineering, 2019, 42(6): e13178. DOI: 10.1111/jfpe.13178.
- [6] da Silva W P, e Silva C M D P S, Gama F J A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. Journal of Food Engineering, 2014, 141: 65–73. DOI: 10.1016/j.jfoodeng.2014.05.010.
- [7] Zhang J K, Zheng X, Xiao H W, et al. Simulation of heat and mass transfer shrinkage and quality of yam slices dried using infrared combined hot air[J]. Transactions of the CSAE, 2024, 40(6): 134–145. DOI: 10.11975/j.issn.1002-6819.202308074.
- [8] Tzempelikos D A, Mitrakos D, Vouros A P, et al. Modelling and simulation of heat and mass transfer during convective drying of cylindrical quince slices[C]//eRA-9. Athens, 2014. (The supplied eight-page conference manuscript.)

附录 A 支撑材料文件列表

表 14 支撑材料及作用

文件	作用
<code>solve_drying.py</code>	四问有限体积求解、首达判定、移动边界与步长检验主程序
<code>write_results.py</code>	四份提交工作簿的生成、格式处理和回读核验
<code>validate_2d.py</code>	二维轴对称端面效应独立核验
<code>run_sensitivity.py</code>	温度、含湿量、扩散、传质和收缩幅度情景分析
<code>make_figures.py</code>	论文科研图的可复现绘制与矢量导出
<code>reproduce.py</code>	按依赖顺序运行复现，工作簿导出为显式选项
<code>grid_study.py</code>	81、161、321、641 节点的空间收敛对照
<code>check_fields.py</code>	场量加密、前段步长与第二三问一致性检查
<code>check_numerics.py</code>	解析模态、界面积分与封闭收缩基准
<code>run_q4_counterfactual.py</code>	附录 4 同物性固定半径对照
<code>make_flowcharts.py</code>	流程图与可编辑源文件
<code>build_paper_data.py</code>	生成正文数值和表格，同步论文代码副本
<code>result1.xlsx</code>	问题一每 1 s、0.1 cm 的温度和水分结果
<code>result2.xlsx</code>	问题二每 1 s、0.1 cm 的温度和水分结果
<code>result3.xlsx</code>	问题三每 60 s、0.1 cm 的水分结果
<code>result4.xlsx</code>	问题四每 60 s、固定位置及动态表面结果
<code>drawio/*.drawio</code>	总体技术路线与移动边界求解流程图源文件
<code>AI_USAGE.md</code>	AI 工具、使用环节、用途及人工核验记录
<code>VERIFY_REPORT.md</code>	数值、工作簿、图形和论文最终验收记录

权威运行入口为 A 题外层 code 目录的 `reproduce.py`。它先生成四问终算场与同物性对照，再运行独立核验、二维验证、灵敏度、绘图和正文数据生成；加入 `--workbooks` 选项可导出四份工作簿。论文内 code 是同步归档副本。内部采用表面加密材料网格，物理半径可随时间变化；程序不含随机过程。论文附录 B 嵌入的主程序与外层运行文件一致。

附录 B 完整源程序

以下为实际运行的四问核心求解程序全文，按源文件顺序单栏连续排版；电子支撑材料中另提供原始 Python 文件。

```
"""Numerical solution for CUMCM 2026 Problem A.

The model uses a node-centred finite-volume discretisation in the radial
coordinate, fully implicit diffusion, Picard iteration for nonlinear
properties, and a material-coordinate transform for uniform radial shrinkage.
"""

from __future__ import annotations

import argparse
import json
import math
import sys
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import numpy as np
from openpyxl import load_workbook
from scipy.interpolate import PchipInterpolator
from scipy.linalg import solve_banded
from scipy.special import expi

ROOT = Path(__file__).resolve().parents[1]
INPUT_DIR = ROOT / "附件"
RESULTS_DIR = ROOT / "results"
H_T = 25.0
H_M = 8.0e-7
R_FIXED = 0.02
NODES = 321
GRID_POWER = 2.0
INITIAL_T = 28.0
INITIAL_C = 2.55
THRESHOLD_C = 0.15

@dataclass
class Environment:
    time_s: np.ndarray
    temperature_c: np.ndarray
    moisture: np.ndarray
```

```

temp_interp: PchipInterpolator
moisture_interp: PchipInterpolator

def values(
    self,
    time_s: float,
    terminal_temp_scale: float = 1.0,
    terminal_moisture_scale: float = 1.0,
) -> tuple[float, float]:
    cutoff = float(self.time_s[-1])
    if time_s <= cutoff:
        return float(self.temp_interp(time_s)), float(self.moisture_interp(time_s))
    return (
        float(self.temperature_c[-1] * terminal_temp_scale),
        float(self.moisture[-1] * terminal_moisture_scale),
    )

@dataclass
class RadiusHistory:
    time_s: np.ndarray
    radius_m: np.ndarray
    interp: PchipInterpolator
    derivative: PchipInterpolator
    monotone_adjustments: int

    def values(self, time_s: float) -> tuple[float, float]:
        bounded = min(max(time_s, float(self.time_s[0])), float(self.time_s[-1]))
        return float(self.interp(bounded)), float(self.derivative(bounded))

def _read_numeric_sheet(path: Path) -> np.ndarray:
    book = load_workbook(path, read_only=True, data_only=True)
    sheet = book.active
    rows: list[list[float]] = []
    for row in sheet.iter_rows(min_row=2, values_only=True):
        if row[0] is None:
            continue
        rows.append([float(value) for value in row])
    return np.asarray(rows, dtype=float)

def load_environment() -> Environment:
    data = _read_numeric_sheet(INPUT_DIR / "附件1.xlsx")
    time_s, temperature_c, moisture = data.T
    if not np.all(np.diff(time_s) > 0):
        raise ValueError("附件1时间列不是严格递增。")

```

```

if not np.all(np.isfinite(data)):
    raise ValueError("附件1含非有限数。")
return Environment(
    time_s=time_s,
    temperature_c=temperature_c,
    moisture=moisture,
    temp_interp=PchipInterpolator(time_s, temperature_c),
    moisture_interp=PchipInterpolator(time_s, moisture),
)

def load_radius_history() -> RadiusHistory:
    data = _read_numeric_sheet(INPUT_DIR / "附件2.xlsx")
    time_s = data[:, 0]
    raw_radius_m = data[:, 1] / 100.0
    if not np.all(np.diff(time_s) > 0):
        raise ValueError("附件2时间列不是严格递增。")
    monotone_radius_m = np.minimum.accumulate(raw_radius_m)
    adjustments = int(np.count_nonzero(np.abs(monotone_radius_m - raw_radius_m) > 1e-12))
    interp = PchipInterpolator(time_s, monotone_radius_m, extrapolate=False)
    return RadiusHistory(
        time_s=time_s,
        radius_m=monotone_radius_m,
        interp=interp,
        derivative=interp.derivative(),
        monotone_adjustments=adjustments,
    )

def property_q1(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    del temperature_c
    rho = np.full_like(moisture, 820.0)
    cp = np.full_like(moisture, 2600.0)
    k = np.full_like(moisture, 0.36)
    safe_c = np.maximum(moisture, 1e-8)
    diffusivity = 7.0e-9 * np.exp(-0.89 / safe_c)
    return rho, cp, k, diffusivity

def property_q23(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    safe_c = np.maximum(moisture, 1e-8)
    rho = 650.0 + 128.0 * safe_c
    cp = 1450.0 + 2736.0 * safe_c / (safe_c + 1.0)
    k = 0.21 + 0.38 * safe_c / (safe_c + 1.0)
    diffusivity = (
        2.4e-3
        * np.exp(-0.45 / safe_c)
    )

```

```

        * np.exp(-3850.0 / (temperature_c + 273.15))
    )
    return rho, cp, k, diffusivity

def property_q4(temperature_c: np.ndarray, moisture: np.ndarray) -> tuple[np.ndarray, ...]:
    safe_c = np.maximum(moisture, 1e-8)
    rho = 760.0 + 90.0 * safe_c
    cp = 1850.0 + 2150.0 * safe_c / (safe_c + 1.0)
    k = 0.12 + 0.20 * safe_c / (safe_c + 1.0)
    diffusivity = (
        4.2e-4
        * np.exp(-0.30 / safe_c)
        * np.exp(-3850.0 / (temperature_c + 273.15))
    )
    return rho, cp, k, diffusivity

def face_interpolate(left: np.ndarray, right: np.ndarray) -> np.ndarray:
    """Linear face interpolation for a smooth state-dependent coefficient."""
    return 0.5 * (left + right)

def moisture_faces(temperature: np.ndarray, moisture: np.ndarray,
                   property_model: Callable, axis: int = 0) -> np.ndarray:
    """Kirchhoff secant flux: integrate D(T_face,C) over the face concentration jump.

     $F_a(C) = C \exp(-a/C) + a \operatorname{Ei}(-a/C)$ , with  $F_a' = \exp(-a/C)$ .
    This avoids arithmetic averaging across the strongly nonlinear dry surface layer.
    """
    left = np.take(moisture, np.arange(moisture.shape[axis] - 1), axis=axis)
    right = np.take(moisture, np.arange(1, moisture.shape[axis]), axis=axis)
    tl = np.take(temperature, np.arange(temperature.shape[axis] - 1), axis=axis)
    tr = np.take(temperature, np.arange(1, temperature.shape[axis]), axis=axis)
    midpoint = np.maximum(0.5 * (left + right), 1e-8)
    exponent = getattr(property_model, 'moisture_exponent', 0.45)
    def primitive(c):
        c = np.maximum(c, 1e-8)
        return c * np.exp(-exponent / c) + exponent * expi(-exponent / c)
    jump = right - left
    close = np.abs(jump) < 1e-5 * midpoint
    secant = np.exp(-exponent / midpoint)
    np.divide(primitive(right) - primitive(left), jump,
              out=secant, where=~close)
    prefactor = property_model(0.5 * (tl + tr), np.ones_like(midpoint))[3] * np.exp(exponent)
    return prefactor * np.maximum(secant, 0.0)

```



```

property_q1.moisture_exponent = 0.89
property_q23.moisture_exponent = 0.45
property_q4.moisture_exponent = 0.30

def node_control_volumes(coordinate: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Return dimensionless/physical radial CV volumes and internal face radii."""
    faces = 0.5 * (coordinate[:-1] + coordinate[1:])
    lower = np.concatenate(([0.0], faces))
    upper = np.concatenate((faces, [coordinate[-1]]))
    volumes = math.pi * (upper**2 - lower**2)
    return volumes, faces

def radial_grid(nodes: int) -> np.ndarray:
    """Surface-graded material grid; resolve the initial sub-millimetre dry layer."""
    return 1.0 - (1.0 - np.linspace(0.0, 1.0, nodes)) ** GRID_POWER

def implicit_radial_step(
    old: np.ndarray,
    storage: np.ndarray,
    conductivity: np.ndarray,
    dt: float,
    coordinate: np.ndarray,
    diffusion_scale: float,
    boundary_conductance: float,
    boundary_value: float,
    advection_velocity: np.ndarray | None = None,
    face_conductivity: np.ndarray | None = None,
) -> np.ndarray:
    """One implicit node-centred radial FVM step on coordinate [0, 1] or [0, R]."""
    count = len(old)
    volumes, faces = node_control_volumes(coordinate)
    delta = np.diff(coordinate)
    face_property = (face_interpolate(conductivity[:-1], conductivity[1:])
                     if face_conductivity is None else face_conductivity)
    conductance = 2.0 * math.pi * faces * face_property * diffusion_scale / delta

    capacity = storage * volumes / dt
    diagonal = capacity.copy()
    lower = np.zeros(count - 1)
    upper = np.zeros(count - 1)
    rhs = capacity * old

    diagonal[:-1] += conductance

```

```

diagonal[1:] += conductance
upper -= conductance
lower -= conductance

diagonal[-1] += boundary_conductance
rhs[-1] += boundary_conductance * boundary_value

if advection_velocity is not None:
    # PDE form:  $\phi_t + u(x) \phi_x = \text{diffusion}$ ;  $u \geq 0$  under shrinkage.
    advective = storage[1:] * volumes[1:] * np.maximum(advection_velocity[1:], 0.0) / delta
    diagonal[1:] += advective
    lower -= advective

matrix = np.zeros((3, count))
matrix[0, 1:] = upper
matrix[1, :] = diagonal
matrix[2, :-1] = lower
return solve_banded((1, 1), matrix, rhs, check_finite=False)

def advance_coupled(
    temperature_old: np.ndarray,
    moisture_old: np.ndarray,
    dt: float,
    radius_m: float,
    radius_rate_m_s: float,
    ambient_temperature_c: float,
    ambient_moisture: float,
    property_model: Callable[[np.ndarray, np.ndarray], tuple[np.ndarray, ...]],
    moving_domain: bool,
    tolerance_temperature: float = 1e-8,
    tolerance_moisture: float = 1e-10,
    max_iterations: int = 30,
) -> tuple[np.ndarray, np.ndarray, int]:
    temperature = temperature_old.copy()
    moisture = moisture_old.copy()
    if moving_domain:
        coordinate = radial_grid(len(temperature_old))
        diffusion_scale = 1.0 / radius_m**2
        boundary_scale = 1.0 / radius_m
        # Uniform material shrinkage:  $v_r = R_{dot} * x$  equals the grid velocity.
        # The relative advection term therefore vanishes in material coordinates.
        shrink_velocity = None
    else:
        coordinate = radius_m * radial_grid(len(temperature_old))
        diffusion_scale = 1.0
        boundary_scale = 1.0

```

```

shrink_velocity = None

for iteration in range(1, max_iterations + 1):
    rho, cp, k, _ = property_model(temperature, moisture)
    heat_storage = rho * cp
    heat_boundary = 2.0 * math.pi * coordinate[-1] * H_T * boundary_scale
    next_temperature = implicit_radial_step(
        old=temperature_old,
        storage=heat_storage,
        conductivity=k,
        dt=dt,
        coordinate=coordinate,
        diffusion_scale=diffusion_scale,
        boundary_conductance=heat_boundary,
        boundary_value=ambient_temperature_c,
        advection_velocity=shrink_velocity,
    )

    _, _, _, diffusivity = property_model(next_temperature, moisture)
    mass_boundary = 2.0 * math.pi * coordinate[-1] * H_M * boundary_scale
    next_moisture = implicit_radial_step(
        old=moisture_old,
        storage=np.ones_like(moisture),
        conductivity=diffusivity,
        dt=dt,
        coordinate=coordinate,
        diffusion_scale=diffusion_scale,
        boundary_conductance=mass_boundary,
        boundary_value=ambient_moisture,
        advection_velocity=shrink_velocity,
        face_conductivity=moisture_faces(next_temperature, moisture, property_model),
    )

    error_t = float(np.max(np.abs(next_temperature - temperature)))
    error_c = float(np.max(np.abs(next_moisture - moisture)))
    temperature = next_temperature
    moisture = next_moisture
    if error_t < tolerance_temperature and error_c < tolerance_moisture:
        break
    else:
        raise RuntimeError(
            f"Picard iteration failed: dT={error_t:.3e}, dC={error_c:.3e}, dt={dt}"
        )

if not np.all(np.isfinite(temperature)) or not np.all(np.isfinite(moisture)):
    raise FloatingPointError("Non-finite state generated.")
if np.min(moisture) < -1e-10 or np.max(moisture) > INITIAL_C + 1e-8:

```

```

        raise FloatingPointError("Moisture left physical bounds.")
    return temperature, moisture, iteration

def _simulate(environment, property_model, end_time_s, dt_s, output_interval_s,
              stop_at_threshold, radius_history=None, terminal_temp_scale=1.0,
              terminal_moisture_scale=1.0, nodes=None, early_dt_s=1.0,
              terminal_temp_offset_c=0.0):
    """Material-coordinate solve; identical first 3 h stepping for Q2 and Q3.

    Normalized CV volumes are fixed dry-solid mass weights. Their sum is pi.
    Physical dry density scales as  $R^{-2}$ , so total dry mass is constant.
    The reported mass balance is water mass per unit dry-solid mass.
    """
    nodes = NODES if nodes is None else nodes
    coordinate = radial_grid(nodes)
    weights = node_control_volumes(coordinate)[0] / math.pi
    temperature = np.full(nodes, INITIAL_T)
    moisture = np.full(nodes, INITIAL_C)
    times, temperatures, moistures, radii = [0.0], [temperature.copy()], [moisture.copy()],
    [R_FIXED]
    iterations = []
    threshold_time = math.nan
    threshold_temperature = threshold_moisture = None
    threshold_radius = math.nan
    time_s, next_output, stop_output_time = 0.0, output_interval_s, math.inf
    maximum_time = float(end_time_s if end_time_s is not None else
                          (radius_history.time_s[-1] if radius_history else 10 * 86400))
    monotone_profiles = center_is_max = True
    integrated_loss = maximum_mass_residual = 0.0
    while time_s < maximum_time - 1e-9:
        requested_dt = min(dt_s, early_dt_s) if time_s < 10800 - 1e-9 else dt_s
        step = min(requested_dt, maximum_time - time_s, next_output - time_s)
        for knot in (10800.0, 14400.0):
            if time_s < knot - 1e-9:
                step = min(step, knot - time_s)
        new_time = time_s + step
        old_t, old_c = temperature.copy(), moisture.copy()
        radius = radius_history.values(new_time)[0] if radius_history else R_FIXED
        ambient_t, ambient_c = environment.values(new_time, terminal_temp_scale,
                                                    terminal_moisture_scale)
        if new_time > environment.time_s[-1]:
            ambient_t += terminal_temp_offset_c
        temperature, moisture, count = advance_coupled(
            old_t, old_c, step, radius, 0.0, ambient_t, ambient_c,
            property_model, moving_domain=radius_history is not None)
        iterations.append(count)

```

```

# Backward-Euler surface flux with the same boundary state as the solver.
boundary_loss = step * 2.0 * H_M / radius * (moisture[-1] - ambient_c)
residual = float(weights @ (moisture - old_c) + boundary_loss)
maximum_mass_residual = max(maximum_mass_residual, abs(residual))
integrated_loss += boundary_loss
time_s = new_time
monotone_profiles &= bool(np.all(np.diff(moisture) <= 2e-9))
center_is_max &= bool(moisture[0] >= np.max(moisture) - 1e-9)
before, after = float(np.max(old_c)), float(np.max(moisture))
if before > THRESHOLD_C >= after and math.isnan(threshold_time):
    fraction = (before - THRESHOLD_C) / (before - after)
    threshold_time = time_s - step + fraction * step
    threshold_temperature = old_t + fraction * (temperature - old_t)
    threshold_moisture = old_c + fraction * (moisture - old_c)
    threshold_radius = radius_history.values(threshold_time)[0] if radius_history else
R_FIXED
    stop_output_time = math.ceil(threshold_time / output_interval_s) *
output_interval_s
    if time_s >= next_output - 1e-8:
        times.append(time_s)
        temperatures.append(temperature.copy())
        moistures.append(moisture.copy())
        radii.append(radius)
        next_output += output_interval_s
    if stop_at_threshold and time_s >= stop_output_time - 1e-8:
        break
if stop_at_threshold and not math.isfinite(threshold_time):
    raise RuntimeError('Threshold was not reached within supplied time horizon')
result = {
    'times_s': np.asarray(times), 'temperature_c': np.asarray(temperatures),
    'moisture': np.asarray(moistures), 'threshold_time_s': threshold_time,
    'threshold_temperature_c': threshold_temperature,
    'threshold_moisture': threshold_moisture, 'last_time_s': time_s,
    'picard_max': int(max(iterations, default=0)),
    'picard_mean': float(np.mean(iterations)),
    'monotone_profiles': bool(monotone_profiles), 'center_is_max': bool(center_is_max),
    'maximum_step_mass_residual': maximum_mass_residual,
    'relative_cumulative_mass_residual': abs(float(weights @ moisture) + integrated_loss -
INITIAL_C) / INITIAL_C,
    'nodes': nodes, 'late_dt_s': dt_s, 'early_dt_s': min(early_dt_s, dt_s),
    'grid_power': GRID_POWER,
    'transport_frame': 'uniform_material_shrinkage' if radius_history else 'fixed',
}
if radius_history:
    result.update(xi=coordinate, radii_m=np.asarray(radii),
                  threshold_radius_m=threshold_radius)
else:

```

```

        result['radius_m'] = coordinate * R_FIXED
    return result

def simulate_fixed(environment, property_model, end_time_s, dt_s, output_interval_s,
                  stop_at_threshold, terminal_temp_scale=1.0,
                  terminal_moisture_scale=1.0, nodes=None, early_dt_s=1.0,
                  terminal_temp_offset_c=0.0):
    return _simulate(environment, property_model, end_time_s, dt_s, output_interval_s,
                    stop_at_threshold, terminal_temp_scale=terminal_temp_scale,
                    terminal_moisture_scale=terminal_moisture_scale, nodes=nodes,
                    early_dt_s=early_dt_s, terminal_temp_offset_c=terminal_temp_offset_c)

def simulate_moving(environment, radius_history, dt_s, output_interval_s,
                  stop_at_threshold=True, terminal_temp_scale=1.0,
                  terminal_moisture_scale=1.0, nodes=None, early_dt_s=1.0,
                  terminal_temp_offset_c=0.0):
    return _simulate(environment, property_q4, None, dt_s, output_interval_s,
                    stop_at_threshold, radius_history=radius_history,
                    terminal_temp_scale=terminal_temp_scale,
                    terminal_moisture_scale=terminal_moisture_scale, nodes=nodes,
                    early_dt_s=early_dt_s, terminal_temp_offset_c=terminal_temp_offset_c)

def extract_fixed_table(
    simulation: dict[str, np.ndarray | float | int | bool],
    requested_times_s: list[float],
    requested_radii_cm: list[float],
    variable: str,
) -> list[list[float]]:
    times = np.asarray(simulation["times_s"])
    radii_m = np.asarray(simulation["radius_m"])
    values = np.asarray(simulation[variable])
    table: list[list[float]] = []
    for requested_time in requested_times_s:
        threshold_time = float(simulation.get("threshold_time_s", math.nan))
        threshold_key = (
            "threshold_temperature_c" if variable == "temperature_c" else "threshold_moisture"
        )
        threshold_profile = simulation.get(threshold_key)
        if (
            threshold_profile is not None
            and math.isfinite(threshold_time)
            and abs(requested_time - threshold_time) < 1e-5
        ):
            profile = np.asarray(threshold_profile)

```

```

        else:
            time_index = int(np.argmin(np.abs(times - requested_time)))
            profile = values[time_index]
            row = [
                float(np.interp(radius_cm / 100.0, radii_m, profile))
                for radius_cm in requested_radii_cm
            ]
            table.append(row)
        return table

def extract_moving_profile(
    simulation: dict[str, np.ndarray | float | int | bool],
    requested_time_s: float,
    requested_radii_cm: list[float],
) -> list[float | None]:
    times = np.asarray(simulation["times_s"])
    xi = np.asarray(simulation["xi"])
    radii_m = np.asarray(simulation["radii_m"])
    moisture = np.asarray(simulation["moisture"])
    threshold_time = float(simulation.get("threshold_time_s", math.nan))
    threshold_profile = simulation.get("threshold_moisture")
    if (
        threshold_profile is not None
        and math.isfinite(threshold_time)
        and abs(requested_time_s - threshold_time) < 1e-5
    ):
        profile = np.asarray(threshold_profile)
        radius = float(simulation["threshold_radius_m"])
    else:
        index = int(np.argmin(np.abs(times - requested_time_s)))
        profile = moisture[index]
        radius = float(radii_m[index])
    values: list[float | None] = []
    for radius_cm in requested_radii_cm:
        position_m = radius_cm / 100.0
        if position_m > radius + 1e-12:
            values.append(None)
        else:
            values.append(float(np.interp(position_m / radius, xi, profile)))
    values.append(float(profile[-1]))
    return values

def save_npz(path: Path, simulation: dict[str, np.ndarray | float | int | bool]) -> None:
    arrays = {key: value for key, value in simulation.items() if isinstance(value, np.ndarray)}
    np.savez_compressed(path, **arrays)

```

```

def json_ready(value):
    if isinstance(value, np.ndarray):
        return value.tolist()
    if isinstance(value, (np.floating, np.integer)):
        return value.item()
    if isinstance(value, float) and math.isnan(value):
        return None
    if isinstance(value, dict):
        return {key: json_ready(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [json_ready(item) for item in value]
    return value

def write_summary(path: Path, payload: dict) -> None:
    path.write_text(
        json.dumps(json_ready(payload), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

def run_all(run_convergence: bool = True) -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    for name in ("q1", "q2", "q3", "q4"):
        (RESULTS_DIR / name).mkdir(parents=True, exist_ok=True)

    environment = load_environment()
    radius_history = load_radius_history()
    data_audit = {
        "environment_rows": len(environment.time_s),
        "environment_time_range_s": [environment.time_s[0], environment.time_s[-1]],
        "environment_sampling_s": np.unique(np.diff(environment.time_s)),
        "temperature_range_c": [
            float(np.min(environment.temperature_c)),
            float(np.max(environment.temperature_c)),
        ],
        "ambient_moisture_range": [
            float(np.min(environment.moisture)),
            float(np.max(environment.moisture)),
        ],
        "radius_rows": len(radius_history.time_s),
        "radius_time_range_s": [radius_history.time_s[0], radius_history.time_s[-1]],
        "radius_range_cm": [
            float(np.min(radius_history.radius_m) * 100.0),
            float(np.max(radius_history.radius_m) * 100.0),

```



```

    ],
    "radius_monotone_adjustments": radius_history.monotone_adjustments,
}

write_summary(RESULTS_DIR / "data_audit.json", data_audit)

q1 = simulate_fixed(
    environment,
    property_q1,
    end_time_s=1800.0,
    dt_s=1.0,
    output_interval_s=1.0,
    stop_at_threshold=False,
)

save_npz(RESULTS_DIR / "q1" / "fields.npz", q1)
q1_times = [100, 300, 600, 900, 1200, 1500, 1800]
table_radaii = [0.0, 0.5, 1.0, 1.5, 2.0]
q1_summary = {
    "table_times_s": q1_times,
    "table_radaii_cm": table_radaii,
    "temperature_table_c": extract_fixed_table(q1, q1_times, table_radaii, "temperature_c"),
    "moisture_table": extract_fixed_table(q1, q1_times, table_radaii, "moisture"),
    "biot_heat": H_T * R_FIXED / 0.36,
    "initial_diffusivity_m2_s": float(property_q1(
        np.array([INITIAL_T]), np.array([INITIAL_C])
    )[3][0]),
    "biot_mass_initial": H_M * R_FIXED / float(property_q1(
        np.array([INITIAL_T]), np.array([INITIAL_C])
    )[3][0]),
    "picard_max": q1["picard_max"],
    "picard_mean": q1["picard_mean"],
    "temperature_bounds_c": [
        float(np.min(q1["temperature_c"])),
        float(np.max(q1["temperature_c"])),
    ],
    "moisture_bounds": [
        float(np.min(q1["moisture"])),
        float(np.max(q1["moisture"])),
    ],
}

write_summary(RESULTS_DIR / "q1" / "summary.json", q1_summary)

q2 = simulate_fixed(
    environment,
    property_q23,
    end_time_s=10800.0,
    dt_s=1.0,
    output_interval_s=1.0,

```

```

        stop_at_threshold=False,
    )
    save_npz(RESULTS_DIR / "q2" / "fields.npz", q2)
    q2_times = [1800, 3600, 5400, 7200, 9000, 10800]
    q2_summary = {
        "table_times_h": [time / 3600.0 for time in q2_times],
        "table_radai_cm": table_radai,
        "temperature_table_c": extract_fixed_table(q2, q2_times, table_radai, "temperature_c"),
        "moisture_table": extract_fixed_table(q2, q2_times, table_radai, "moisture"),
        "picard_max": q2["picard_max"],
        "picard_mean": q2["picard_mean"],
        "monotone_profiles": q2["monotone_profiles"],
        "center_is_max": q2["center_is_max"],
    }
    write_summary(RESULTS_DIR / "q2" / "summary.json", q2_summary)

    q3 = simulate_fixed(
        environment,
        property_q23,
        end_time_s=None,
        dt_s=10.0,
        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    save_npz(RESULTS_DIR / "q3" / "fields.npz", q3)
    q3_end = float(q3["threshold_time_s"])
    q3_report_times = list(np.arange(6 * 3600, math.floor(q3_end / (6 * 3600)) * 6 * 3600 + 1,
        6 * 3600))
    if not q3_report_times or abs(q3_report_times[-1] - q3_end) > 1e-6:
        q3_report_times.append(q3_end)
    q3_summary = {
        "drying_time_s": q3_end,
        "drying_time_h": q3_end / 3600.0,
        "output_end_s": q3["last_time_s"],
        "table_times_h": [time / 3600.0 for time in q3_report_times],
        "table_radai_cm": table_radai,
        "moisture_table": extract_fixed_table(q3, q3_report_times, table_radai, "moisture"),
        "center_is_max": q3["center_is_max"],
        "monotone_profiles": q3["monotone_profiles"],
        "picard_max": q3["picard_max"],
        "picard_mean": q3["picard_mean"],
    }

    q4 = simulate_moving(
        environment,
        radius_history,
        dt_s=10.0,

```

```

        output_interval_s=60.0,
        stop_at_threshold=True,
    )
    save_npz(RESULTS_DIR / "q4" / "fields.npz", q4)
    q4_end = float(q4["threshold_time_s"])
    q4_report_times = list(np.arange(6 * 3600, math.floor(q4_end / (6 * 3600)) * 6 * 3600 + 1,
        6 * 3600))
    if not q4_report_times or abs(q4_report_times[-1] - q4_end) > 1e-6:
        q4_report_times.append(q4_end)
    q4_table_radii = [0.0, 0.5, 1.0]
    q4_summary = {
        "drying_time_s": q4_end,
        "drying_time_h": q4_end / 3600.0,
        "output_end_s": q4["last_time_s"],
        "table_times_h": [time / 3600.0 for time in q4_report_times],
        "table_radii_cm": q4_table_radii,
        "columns": q4_table_radii + ["药材表面"],
        "moisture_table": [
            extract_moving_profile(q4, time, q4_table_radii) for time in q4_report_times
        ],
        "radius_at_end_cm": float(radius_history.values(q4_end)[0] * 100.0),
        "center_is_max": q4["center_is_max"],
        "monotone_profiles": q4["monotone_profiles"],
        "picard_max": q4["picard_max"],
        "picard_mean": q4["picard_mean"],
    }

    if run_convergence:
        q3_fine = simulate_fixed(
            environment,
            property_q23,
            end_time_s=None,
            dt_s=5.0,
            output_interval_s=60.0,
            stop_at_threshold=True,
        )
        q4_fine = simulate_moving(
            environment,
            radius_history,
            dt_s=5.0,
            output_interval_s=60.0,
            stop_at_threshold=True,
        )
        q3_summary["time_step_check"] = {
            "dt10_s_time_h": q3_end / 3600.0,
            "dt5_s_time_h": float(q3_fine["threshold_time_s"]) / 3600.0,
            "relative_difference": abs(

```

```

        float(q3_fine["threshold_time_s"]) - q3_end
    ) / float(q3_fine["threshold_time_s"]),
}

q4_summary["time_step_check"] = {
    "dt10_s_time_h": q4_end / 3600.0,
    "dt5_s_time_h": float(q4_fine["threshold_time_s"]) / 3600.0,
    "relative_difference": abs(
        float(q4_fine["threshold_time_s"]) - q4_end
    ) / float(q4_fine["threshold_time_s"]),
}

for name, result in [('q1', q1), ('q2', q2), ('q3', q3), ('q4', q4)]:
    write_summary(RESULTS_DIR / name / 'diagnostics.json', {
        key: value for key, value in result.items() if not isinstance(value, np.ndarray)
        and value is not None})
write_summary(RESULTS_DIR / "q3" / "summary.json", q3_summary)
write_summary(RESULTS_DIR / "q4" / "summary.json", q4_summary)

print(json.dumps(
    {
        "q1_surface_temperature_1800_c": q1_summary["temperature_table_c"][-1][-1],
        "q1_center_moisture_1800": q1_summary["moisture_table"][-1][0],
        "q2_center_temperature_3h_c": q2_summary["temperature_table_c"][-1][0],
        "q2_center_moisture_3h": q2_summary["moisture_table"][-1][0],
        "q3_drying_time_h": q3_summary["drying_time_h"],
        "q4_drying_time_h": q4_summary["drying_time_h"],
        "q4_radius_at_end_cm": q4_summary["radius_at_end_cm"],
    },
    ensure_ascii=False,
    indent=2,
))

def main() -> None:
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser()
    parser.add_argument("--skip-convergence", action="store_true")
    args = parser.parse_args()
    run_all(run_convergence=not args.skip_convergence)

if __name__ == "__main__":
    main()

```